

Enterprise-Grade Model Context Protocol: Architecting a Multi-Tenant Gateway with Dynamic RBAC, Multi-Backend Integration, and Defense-in-Depth Observability

Author: Raheem Mohammed **Date:** September 2026 **Document Classification:** Technical Whitepaper -- Academic & Portfolio Submission Ready **Terminology Note:** This paper uses vendor-neutral and anonymized terminology. All open-source protocols, standards, and core technology names are retained as-is.

Abstract

The Model Context Protocol (MCP) has emerged as the de facto standard for connecting large language models (LLMs) and AI agents to external tools and data sources. However, deploying MCP servers in regulated enterprise environments exposes fundamental gaps in the open-source specification: the protocol defines no authentication, no authorization, no audit trail, and no rate governance. This paper presents the architecture of an Enterprise MCP Gateway that closes these gaps through a layered, production-hardened design. The gateway mediates access to 39 tools across 7 heterogeneous backend systems -- including project management (Jira), graph databases (Neo4j), analytical data warehouses (Teradata), document databases (DocumentDB), business rule engines, reference data services, and an enterprise LLM orchestrator -- for multiple consumer classes: AI agents, IDE-based developer copilots (Kiro IDE), REST API clients, and batch ETL processes. The security perimeter implements three-mode JWT authentication, Active Directory group-based role-based access control (RBAC) with five graduated permission tiers, seven composable security guardrails enforced per request, and AES-256-GCM envelope encryption via AWS KMS for credential storage. Observability is addressed through three parallel audit pipelines writing to DocumentDB, structured log sanitization with SHA-256 credential fingerprinting, and two-layer sliding-window rate limiting (per-consumer and per-tool-group). All governance parameters -- rate limits, tool availability, guardrail thresholds, and backend routing -- are dynamically configurable via DocumentDB without

redeployment, propagating across horizontally-scaled pods within 30 seconds. The system has been deployed across four environments (sandbox, development, UAT, production) on Amazon EKS with CI/CD-driven feature flags, and is presented to a Solution Review and Planning Board (SRPB) for enterprise architecture approval. This paper maps the gateway's architectural patterns to graduate-level learning outcomes in natural language processing, deep learning systems, and distributed AI architectures, demonstrating applied mastery of the infrastructure that enables responsible enterprise AI deployment.

Executive Summary

As organizations accelerate AI adoption, the gap between experimental AI prototypes and production-grade enterprise systems widens. The Model Context Protocol, published by Anthropic in 2024, standardizes how AI models invoke external tools -- but the specification intentionally omits security, governance, and observability, leaving these concerns to implementers. This paper documents a production implementation that fills those gaps.

The Enterprise MCP Gateway described herein is not a proxy. It is a multi-tenant, multi-backend API gateway that enforces identity verification, fine-grained authorization, input validation, rate governance, credential management, and comprehensive audit logging on every tool invocation. It supports four distinct consumer types through three authentication modes, routes requests to seven backend systems through a unified interface, and provides runtime-configurable governance without redeployment.

Key metrics from the reference implementation:

Metric	Value
MCP tools exposed	39 (across 4 tool groups + 2 built-in)
Backend systems integrated	7
Authentication modes	3 (API Key + JWT, Kiro RBAC, Legacy)
Security guardrails per request	7 composable checks
Audit pipelines	3 parallel (domain-specific, generic, access)
Rate limiting layers	2 (per-consumer + per-tool-group)

Encryption standard	AES-256-GCM with KMS envelope encryption
Cache layers	3 (Redis distributed, disk-backed reference, in-memory registry)
Deployment environments	4 (sandbox, dev, UAT, prod)
Feature flags	6 CI/CD-driven toggles
DocumentDB governance collections	11
RBAC roles	5 AD-group-mapped tiers

Section 1: Introduction and Enterprise Architectural Challenges

1.1 The MCP Protocol Gap

The Model Context Protocol (MCP) defines a JSON-RPC 2.0 interface for tool discovery (`tools/list`) and tool invocation (`tools/call`) between AI model hosts and tool servers. The protocol specification addresses transport (stdio, HTTP with Server-Sent Events) and message framing, but explicitly defers security to the deployment layer. In its reference implementation, an MCP server exposes all registered tools to any connected client with no authentication, no authorization checks, no rate limiting, and no audit trail.

This design is appropriate for local development where a single developer runs a single MCP server against their own resources. It is insufficient for enterprise deployment where:

- **Multiple consumer types** (AI agents, IDE copilots, REST services, batch jobs) share a single gateway
- **Heterogeneous backends** (project management, graph databases, analytical warehouses, business rule engines) have different security requirements
- **Regulatory compliance** demands audit trails for every data access and mutation
- **Shared credentials** (service accounts, API tokens) must be attributed to individual users
- **Backend rate limits** (e.g., Jira's 60 requests/minute burst limit) must be

respected across all consumers

1.2 Enterprise Requirements

The Enterprise MCP Gateway was designed to satisfy the following non-functional requirements, derived from common enterprise architecture principles:

1. **Zero-trust identity:** Every request must carry a verified identity. Header-declared identity is overridden by cryptographically verified JWT claims when available.
2. **Least-privilege authorization:** Each consumer receives only the tools and projects their role permits. Wildcard access (*) is never stored -- it is expanded to explicit lists at registration time.
3. **Write attribution:** Mutations must be attributed to the individual user, not a shared service account. This requires personal credentials (e.g., personal access tokens) for write operations.
4. **Defense-in-depth:** Security is enforced at multiple layers -- authentication middleware, guardrail pipeline, tool-level decorators, and backend-level restrictions -- so that a failure in any single layer does not compromise the system.
5. **Audit completeness:** Every tool invocation (successful, failed, rate-limited, or guardrail-blocked) is recorded with caller identity, credential fingerprint, duration, and outcome.
6. **Runtime configurability:** Governance parameters must be changeable without redeployment, propagating across all pods within a bounded window.
7. **Graceful degradation:** The gateway must remain available when individual backends or governance stores are temporarily unavailable, with clear fail-open vs. fail-closed policies per component.

1.3 System Scope

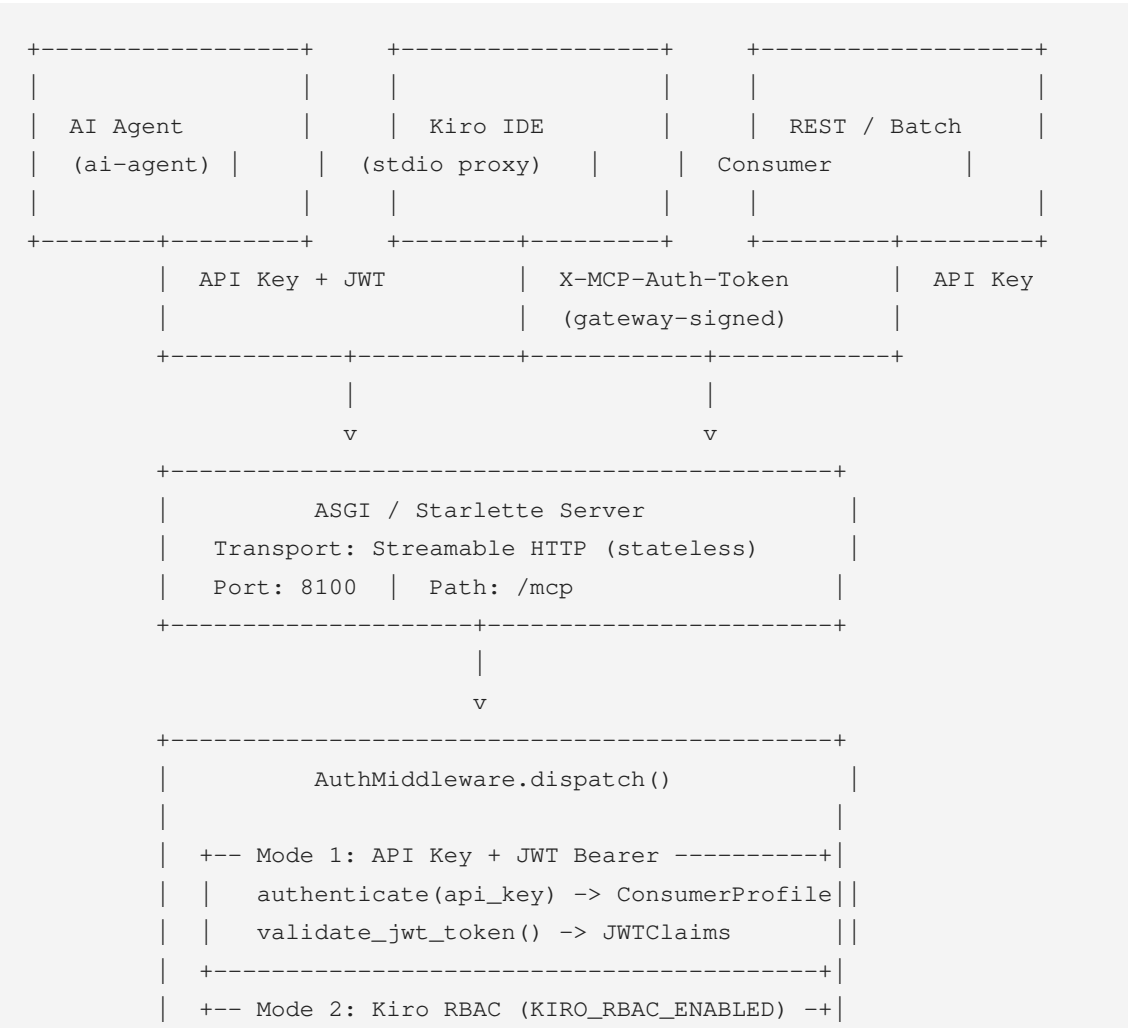
The gateway mediates access to the following backend systems:

Backend	Protocol	Tool Count	Access Pattern
Jira (Project Management)	REST API / Enterprise LLM Orchestrator JSON-RPC	12	Read + Write
Business Rule Engine (Reference Data Service)	REST API	15	Read + Write

Neo4j (Knowledge Graph)	Bolt protocol	3	Read-only
Teradata (Analytical Data Warehouse)	MCP sidecar (Streamable HTTP)	7	Read-only
DocumentDB (Document Database)	MongoDB wire protocol	Indirect (config, audit)	Read + Write
Enterprise SQL Server	ODBC	Indirect (BRE lookup)	Read-only
Enterprise LLM Orchestrator	JSON-RPC over HTTPS	Indirect (Jira routing)	Read + Write

1.4 Architectural Component Diagram

The following diagram illustrates the end-to-end request path from client through every gateway layer to the backend systems. Each box represents a discrete architectural component; arrows indicate data flow direction.



```

| | X-MCP-Auth-Token -> validate_jwt_token | |
| | Build scoped ConsumerProfile from claims | |
| +-----+ |
| +-- Mode 3: Kiro Legacy (auto-whitelist) ---+ |
| | Wildcard ConsumerProfile (all tools) | |
| +-----+ |
| |
| Output: CallerIdentity + ContextVars |
| (user_id, groups, consumer, creds) |
+-----+
|
| v
+-----+
| Guardrail Pipeline (7 Checks) |
| |
| [0] Tool Active Check (registry lookup) |
| [1] Tool Authorization (allowed_tools) |
| [2] User Write Auth (AD group gate) |
| [3] Write Credential Attribution (PAT req.) |
| [4] JQL Input Validation (injection block) |
| [5] SQL Read-Only Validation (DDL/DML block) |
| [6] Project Scope Enforcement |
| [7] Write Rate Limiting (5 writes/600s) |
| |
| All thresholds: DocumentDB-backed (30s TTL) |
+-----+
|
| v
+-----+
| Two-Layer Rate Limiter |
| |
| Layer 1: Per-consumer sliding window |
| (from RBAC role: 10-30 calls/min) |
| Layer 2: Per-tool-group global limit |
| (from DocumentDB: default 59/min) |
| |
| Bypass: user-provided project-management |
| or orchestrator credential |
+-----+
|
| v
+-----+
| Backend Dispatcher |
| |
| Tool group routing: |
| jira_* -> Jira REST / LLM Orchestrator |
| bre_* -> Business Rule Engine (RDS) |

```

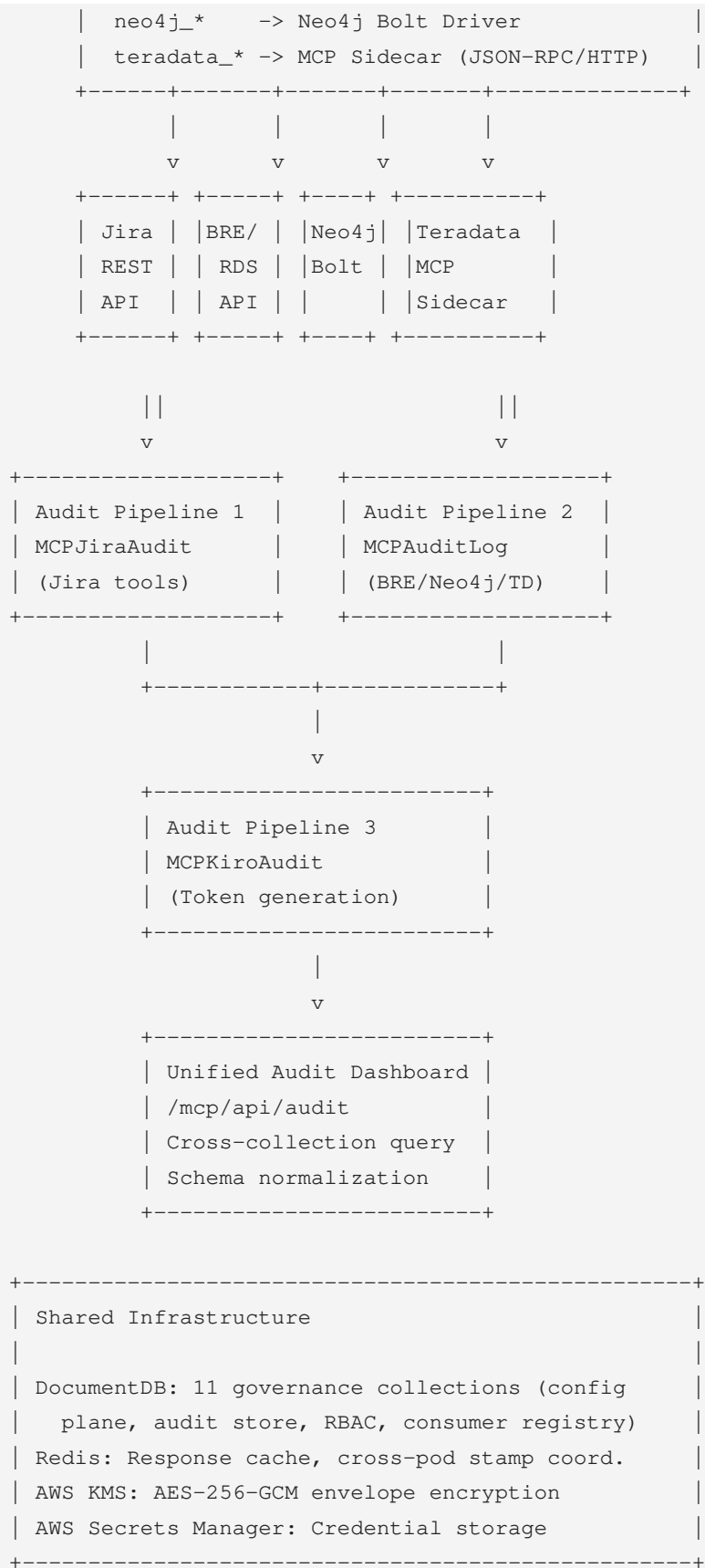


Figure 1. Enterprise MCP Gateway -- end-to-end request flow from client authentication through guardrail enforcement, rate limiting, backend dispatch, and dual audit logging. All governance components read dynamic configuration from DocumentDB with bounded-TTL caching.

Section 2: Multi-Mode Security Perimeter

2.0 Threat Model

This subsection defines the primary threat vectors addressed by the Enterprise MCP Gateway's security architecture and maps each threat to the specific controls that mitigate it. The threat model follows a STRIDE-informed approach, focusing on the three vectors most relevant to an AI tool gateway operating in a multi-tenant enterprise environment.

T1: Identity Spoofing

Threat Description: A malicious or compromised consumer forges identity headers (X-MCP-User-ID, X-MCP-Username, X-MCP-User-Groups) to impersonate a higher-privileged user, bypass write authorization gates, or attribute mutations to another individual.

Attack Surface: MCP's JSON-RPC protocol carries no native identity. The gateway relies on HTTP headers for identity propagation. Any consumer with network access to the gateway endpoint could inject arbitrary identity headers.

Mitigating Controls:

Control	Layer	Mechanism
JWT signature verification	AuthMiddleware	<code>validate_jwt_token()</code> cryptographically verifies the token signature using <code>JWT_SECRET</code> or the IdP's signing key. Verified JWT claims override all header-declared identity fields. A forged header is discarded when a valid JWT is present.

Gateway-signed Kiro tokens	AuthMiddleware (Mode 2)	Kiro IDE tokens are signed by the gateway itself (HS256). The token embeds <code>user_id</code> , <code>groups</code> , <code>role</code> , <code>allowed_tools</code> , and <code>projects</code> at issuance time. Identity is extracted from the enterprise IdP's SSO cookie during token generation -- not from user-supplied headers.
Credential fingerprinting	Audit Pipeline	<code>secret_fingerprint()</code> computes a SHA-256 hash (first 12 hex chars) of each credential, creating a non-reversible correlation ID. Even if identity headers are spoofed, the credential fingerprint in the audit trail links the request to the actual credential used, enabling forensic detection.
Consumer type enforcement	<code>is_jwt_required_for_consumer()</code>	When <code>JWT_AUTH_ENABLED=true</code> , consumers of type <code>ai-agent</code> MUST present a valid JWT. Absence of a token results in a 401 rejection -- header-only identity is insufficient.

Residual Risk: In Kiro Legacy mode (`KIRO_RBAC_ENABLED=false`), identity headers are trusted without verification. This mode is intended for local development only and is disabled in all deployed environments via CI/CD variable.

T2: Adversarial Query and Prompt Injection

Threat Description: An AI agent, acting on adversarial or manipulated user input, generates tool invocation parameters containing injection payloads. For Jira tools, this manifests as JQL injection (e.g., `project = PROJ; DROP TABLE`). For Teradata tools, this manifests as SQL injection attempting DDL/DML operations (e.g., `SELECT 1; DELETE FROM accounts`).

Attack Surface: LLM-generated tool parameters are inherently untrusted. The model may be influenced by prompt injection attacks embedded in documents, issue descriptions, or user messages. The gateway cannot distinguish between intentional and adversarial model outputs -- it must validate all inputs structurally.

Mitigating Controls:

Control	Layer	Mechanism
JQL Input Validation (Guardrail 4)	Guardrail Pipeline	Regex-based detection of injection patterns: semicolons followed by SQL keywords (; DROP, ; DELETE), comment sequences (--, /* */), hexadecimal escape sequences (0x), Unicode escape sequences. Enforces a 2,000-character input limit.
SQL Read-Only Validation (Guardrail 5)	Guardrail Pipeline	For Teradata tools: requires query to begin with SELECT or WITH; blocks all DDL keywords (CREATE, ALTER, DROP, TRUNCATE) and DML keywords (INSERT, UPDATE, DELETE, MERGE); enforces single-statement (no semicolons); 5,000-character limit.
Four-layer Teradata read-only enforcement	Backend + Sidecar + Guardrail + DB	(1) Database account has no write grants, (2) sidecar profile is readonly, (3) gateway SQL guardrail blocks DDL/DML, (4) row cap limits result size. An injection must bypass all four layers simultaneously.
Project Scope Enforcement (Guardrail 6)	Guardrail Pipeline	Issue keys and JQL project references are parsed and validated against the consumer's authorized project list. An injected query referencing unauthorized projects is blocked before reaching the backend.

Residual Risk: JQL validation uses pattern matching, not a full parser. A sufficiently novel injection pattern not covered by the current regex set could bypass validation. Mitigation: the guardrail regex patterns are stored in DocumentDB and updatable at runtime without redeployment.

T3: Backend Resource Exhaustion

Threat Description: A single consumer (or coordinated group of consumers) overwhelms a shared backend system by issuing tool calls at a rate exceeding the backend's capacity. For Jira, this triggers HTTP 429 rate limiting that affects all consumers. For Teradata, this could saturate the analytical query pool. For Neo4j, this could exhaust the connection pool.

Attack Surface: The MCP protocol provides no built-in flow control. A misbehaving AI agent can issue hundreds of tool calls per minute. Multiple consumers sharing the gateway's service account credentials amplify the aggregate load on backends with per-account rate limits.

Mitigating Controls:

Control	Layer	Mechanism
Per-consumer rate limiting (Layer 1)	Rate Limiter	Sliding-window limit per <code>consumer_id:tool_group</code> pair. Limit derived from RBAC role (10-30 calls/min). Prevents any single consumer from monopolizing backend capacity.
Per-tool-group global rate limiting (Layer 2)	Rate Limiter	Aggregate sliding-window limit across all consumers per tool group. Default 59/min for Jira (one under the 60-call burst limit). Protects the backend from collective overload even when individual consumers are within their limits.
Write rate limiting (Guardrail 7)	Guardrail Pipeline	Separate per-consumer sliding window for write operations (5 writes/600s). Prevents mutation storms independent of read rate allocation.
User-credential bypass	Rate Limiter	Consumers providing their own PAT bypass the gateway's rate limiter because they consume their own API quota, not the shared service account's. This shifts the exhaustion risk to the individual user's own rate allocation.
Maintenance windows	Maintenance System	DocumentDB-backed blackout periods that block all tool calls to a specific backend during planned maintenance, preventing cascading failures during degraded backend states.
Neo4j auto-limit injection	Neo4j Client	<code>_apply_limit()</code> injects a <code>LIMIT</code> clause into Cypher queries that lack one, preventing unbounded result sets from exhausting memory or connection pool resources.

Residual Risk: The in-memory rate limiter does not share state across pods. In a 4-replica deployment, the effective global rate limit is up to 4x the configured value. Mitigation: the configured limit (59/min) accounts for this multiplier against Jira's actual 60/min burst limit.

2.1 Authentication Architecture

The gateway implements three authentication modes within a single ASGI middleware, selected automatically based on request characteristics:

Mode 1: API Key + JWT Bearer Token Service-to-service consumers (AI agents, REST clients) present an `X-MCP-API-Key` header that identifies the application and determines its tool/project scope from a consumer registry. When `JWT_AUTH_ENABLED=true`, the consumer must additionally present an `Authorization: Bearer <token>` header containing a JWT signed by the enterprise Identity Provider (IdP). The JWT identity (`user_id`, `username`, `email`, `groups`) cryptographically overrides

any header-declared identity, establishing zero-trust verification.

The JWT validator is IdP-agnostic: claim field names are configurable via environment variables (`JWT_CLAIM_USER_ID`, `JWT_CLAIM_USERNAME`, `JWT_CLAIM_EMAIL`, `JWT_CLAIM_GROUPS`), each accepting comma-separated candidate field names tried in order. Algorithm negotiation supports HS512, HS256, and RS256 simultaneously, enabling gradual migration between signing algorithms.

Mode 2: Kiro IDE RBAC (Gateway-Signed Tokens) IDE-based developer copilots (Kiro IDE) cannot carry API keys in the MCP stdio transport. When `KIRO_RBAC_ENABLED=true`, the gateway requires an `X-MCP-Auth-Token` header containing a JWT signed by the gateway itself. This token is obtained through a browser-based flow:

1. The user authenticates via the enterprise IdP (OAuth/SSO cookie)
2. The user visits the gateway's Kiro Access portal
3. The gateway extracts the user's identity from the SSO cookie
4. Active Directory groups are resolved server-side from the enterprise user profile database (no direct LDAP required -- critical for containerized environments where LDAP access is typically blocked)
5. The user's AD groups are matched against RBAC role definitions stored in DocumentDB
6. A scoped JWT is signed with the gateway's secret, embedding the user's `allowed_tools`, `projects`, `rate_limit`, and `role`
7. The token is returned for use in the IDE's MCP configuration

This design eliminates per-user provisioning: AD group membership automatically grants appropriate access.

Mode 3: Kiro IDE Legacy (Auto-Whitelist) When `KIRO_RBAC_ENABLED=false`, Kiro IDE consumers receive wildcard access with no authentication. This mode exists for backward compatibility and local development only.

Identity Propagation Regardless of authentication mode, the middleware constructs a `CallerIdentity` object containing the verified `app_name`, `user_id`, `username`, `user_email`, `consumer profile`, `user_groups`, and `credential_fingerprint`. This identity is stored in a Python `contextvars.ContextVar` for async-safe access by downstream audit loggers, rate limiters, and tool handlers -- ensuring identity is available throughout the request lifecycle without explicit parameter passing.

2.2 Active Directory Group-Based RBAC

The RBAC system maps Active Directory groups to MCP roles with graduated permissions:

Role	Tool Access	Project Scope	Rate Limit	Typical AD Group
Admin	All tools (39)	All projects	30 calls/min	Platform administrators
Developer	All tools (39)	All projects	15 calls/min	Development team members
Supervisor	All tools (39)	All projects	20 calls/min	Team leads and supervisors
Reader	27 read-only tools	6 specific projects	10 calls/min	Standard platform users
User (default)	Configurable subset	Configurable	15 calls/min	General authenticated users

Role definitions are stored in a dedicated DocumentDB collection (`mcp_rbac_roles`) with one document per AD group. The collection supports full CRUD operations via the admin dashboard, with cache invalidation on every write to ensure changes take effect within 5 minutes across all pods.

A profile override layer (`mcp_kiro_profiles`) allows per-user customization: a user can be granted elevated access beyond their AD group role, or explicitly denied access by setting `is_active=false`. The override layer is checked before AD group matching, providing administrative flexibility without modifying group membership.

2.3 Seven Composable Security Guardrails

Every tool invocation passes through a pipeline of seven independent security checks, executed in sequence. Each check returns either `None` (pass) or a structured error dictionary (block). The pipeline is implemented in `enforce_guardrails()` and all thresholds are dynamically configurable via DocumentDB with environment-variable fallback:

Guardrail 0 -- Tool Active Check: Verifies the requested tool is enabled in the tool registry. Disabled tools return a `tool_disabled` error. This provides a runtime kill switch for any tool without code changes.

Guardrail 1 -- Tool Authorization: Validates the requested tool against the consumer's `allowed_tools` whitelist. A wildcard (*) grants access to all tools;

otherwise, exact name matching is required.

Guardrail 2 -- User Write Authorization: For AI-agent consumers invoking write tools, this check requires the user's Active Directory groups to include at least one write-authorized group. AD groups are resolved from JWT claims first, then from the enterprise user profile database as a fallback.

Guardrail 3 -- Write Credential Attribution: For AI-agent consumers invoking write tools, this check requires a personal access token (PAT) with source `request_header` or `profile -- not server_config`. This ensures that mutations are attributed to the individual user in the backend system's audit trail, preventing the "shared service account" anti-pattern where all AI-agent writes appear as a single identity.

Guardrail 4 -- JQL Input Validation: For tools accepting Jira Query Language (JQL) input, this check blocks SQL injection patterns (`;``DROP`, `--`, `/* */`, hexadecimal and Unicode escape sequences) and enforces a 2,000-character limit.

Guardrail 5 -- SQL Read-Only Validation: For Teradata tools, this check ensures only single `SELECT` or `WITH` statements are submitted. All DDL (`CREATE`, `ALTER`, `DROP`) and DML (`INSERT`, `UPDATE`, `DELETE`) keywords are blocked. A 5,000-character limit prevents excessively complex queries.

Guardrail 6 -- Project Scope Enforcement: Validates that the consumer is authorized to access the referenced Jira projects. Issue keys are parsed for their project prefix (e.g., `PROJ-123` extracts `PROJ`), and JQL strings are scanned for project references. Unauthorized project access is blocked with a descriptive error.

Guardrail 7 -- Write Rate Limiting: A separate per-consumer sliding window for write operations (default: 5 writes per 600 seconds), independent of the general rate limiter. This prevents a single consumer from making excessive mutations regardless of their read rate allocation.

2.4 AES-256-GCM Envelope Encryption

User credentials (personal access tokens, API keys) stored in DocumentDB are encrypted using AWS KMS envelope encryption:

1. AWS KMS generates a 256-bit Data Encryption Key (DEK) per encryption operation
2. The DEK encrypts the plaintext token using AES-256-GCM (authenticated encryption with associated data)
3. KMS encrypts the DEK with the Customer Master Key (CMK), which never leaves the HSM

4. The gateway stores three values: `encrypted_token` (ciphertext), `encrypted_key` (encrypted DEK), and `nonce` (12-byte GCM initialization vector)

To decrypt, the gateway sends the encrypted DEK to KMS (which logs the operation in CloudTrail), receives the plaintext DEK, and decrypts the token locally. This design provides:

- **Encryption at rest:** No plaintext credentials in DocumentDB
 - **HSM-backed key management:** The master key never leaves the KMS hardware security module
 - **Audit trail:** Every decryption is logged in AWS CloudTrail
 - **Transparent key rotation:** KMS handles CMK rotation without re-encrypting stored data
 - **Graceful fallback:** Local development uses Fernet symmetric encryption when KMS is unavailable
-

Section 3: Governance, Rate Limiting, and Multi-Pipeline Audit Observability

3.1 Two-Layer Sliding-Window Rate Limiting

The gateway implements a two-layer rate limiter protecting both individual consumers and shared backend systems:

Layer 1 -- Per-Consumer Rate Limiting (Inner): Each consumer's rate limit is embedded in their RBAC role (e.g., 15 calls/min for developers, 30 calls/min for admins). The limiter tracks timestamps per `consumer_id:tool_group` pair in a sliding window. When a consumer exceeds their allocation, the gateway returns a structured error with `retry_after_seconds` and `limit_type: "consumer"`.

Layer 2 -- Global Per-Tool-Group Rate Limiting (Outer): Each tool group (jira, brf, neo4j, teradata) has a global rate limit loaded from DocumentDB (default: 59 calls/min for Jira -- deliberately one under Jira's 60-call burst limit to provide a safety margin). This limit applies across all consumers combined, protecting the backend from aggregate overload.

Both layers use a sliding-window algorithm with in-memory timestamp tracking. The `@rate_limited` decorator wraps every tool function, checking Layer 1 then Layer 2 in sequence. Both must pass for the call to proceed.

Formal Rate Limiting Algorithm:

The gateway employs a timestamp-based sliding window algorithm. For a given window of duration T seconds, the effective request count at time t is computed as:

$$N_{\text{window}}(t) = C_{\text{current}} + C_{\text{previous}} * (1 - (t - t_{\text{start}}) / T) \leq L$$

Where:

- $N_{\text{window}}(t)$ is the weighted request count at evaluation time t
- C_{current} is the number of requests in the current window partition $[t_{\text{start}}, t_{\text{start}} + T)$
- C_{previous} is the number of requests in the immediately preceding window partition $[t_{\text{start}} - T, t_{\text{start}})$
- t_{start} is the start time of the current window partition
- T is the window duration in seconds (default: 60s)
- L is the rate limit threshold (per-consumer from RBAC role, or per-tool-group from DocumentDB)

The term $(1 - (t - t_{\text{start}}) / T)$ provides a linear decay weight on the previous window's count, approximating a true sliding window without storing per-request timestamps for the entire history. When t is at the start of the current window, the previous window's full count is included; as t approaches the end, the previous window's contribution decays to zero.

In the gateway's implementation, individual timestamps are stored (rather than bucketed counts) and pruned against the window boundary, providing exact sliding-window semantics:

```
timestamps[key] = [ts for ts in timestamps[key] if ts > now - T]
if len(timestamps[key]) >= L:
    return RATE_LIMITED (with retry_after = T - (now - timestamps[key][0]) + 1)
```

The dual evaluation proceeds as:

```
if N_consumer(t) >= L_consumer:      # Layer 1: per-consumer limit
    return BLOCKED (limit_type="consumer")
if N_group(t) >= L_group:            # Layer 2: global tool-group limit
    return BLOCKED (limit_type="global")
record(t, consumer_id, group)        # Both passed -- record and proceed
```

The generic audit pipeline implements a **fail-open** design: if DocumentDB becomes unavailable, an `_audit_disabled` flag is set for the process lifetime, allowing tool calls to proceed without audit persistence. This design prioritizes tool availability over audit completeness -- the structured logger (forwarded to the centralized log aggregator) serves as a secondary audit path.

Pipeline 3 -- Access Audit (Token Generation): Every Kiro token generation attempt -- successful or failed -- is recorded in the `MCPKiroAudit` collection. Records capture: event type, user identity, AD groups at issuance time, scope granted (or denial reason), client IP, user agent, and gateway environment. This pipeline enables compliance reporting on who accessed the gateway, when, and with what permissions.

Unified Audit Dashboard: A dashboard API reads from all three collections, normalizing schemas and merge-sorting by timestamp. Aggregation endpoints provide real-time usage metrics (calls per minute/hour/day), breakdowns by user, tool, and status, and historical trend analysis.

3.3 Log Sanitization and Credential Fingerprinting

All audit records and structured logs pass through a sanitization layer:

- **Recursive redaction:** Fields matching sensitive patterns (authorization, `api_key`, `token`, `pat`, `password`, `secret`, `cookie`, `credential`, `session`) are replaced with a partial representation showing the first 4 and last 4 characters (e.g., `eyJh...2bcd`), maintaining traceability without exposing the full secret.
- **Output bounding:** String values are capped at 500 characters, collections at 50 items, preventing log bloat from large API responses.
- **Credential fingerprinting:** A SHA-256 hash (first 12 hex characters) provides a non-reversible correlation identifier for each credential. The same PAT produces the same fingerprint across all audit entries, enabling usage tracking without storing or transmitting the secret.

3.4 DocumentDB as Dynamic Configuration Plane

A distinctive architectural choice is the use of DocumentDB as the runtime configuration store for all governance parameters. Eleven collections serve as the dynamic configuration plane:

Collection	Governs	Cache TTL
------------	---------	-----------

mcp_consumers	API key registry, tool/project scope	300s
mcp_rbac_roles	AD group to MCP role mappings	300s
mcp_kiro_profiles	Per-user profile overrides	300s
mcp_tool_registry	Tool metadata, enable/disable	300s
mcp_tool_groups	Tool group rate limits	300s
mcp_jira_projects	Project scope definitions	300s
mcp_gateway_config	Guardrails, backend mode, cache TTL, write AD groups	30s
mcp_brf_defaulting_rules	Cohort-based field suggestion rules	300s
MCPJiraAudit	Jira tool audit trail	N/A (write)
MCPAuditLog	Generic tool audit trail	N/A (write)
MCPKiroAudit	Kiro access audit trail	N/A (write)

Every configuration collection is backed by an in-memory cache with a bounded TTL. Environment variables serve as fallback defaults when DocumentDB is unavailable. This design enables:

- **Runtime tuning:** An administrator changes a rate limit in the dashboard; all pods pick it up within 30 seconds
- **Feature flagging:** A tool is disabled in the registry; all pods stop serving it within 5 minutes
- **Graceful degradation:** DocumentDB outage does not crash the gateway; stale cached values or env-var defaults take over

3.5 In-Memory Metrics and Operational Visibility

A thread-safe in-memory metrics collector (FIFO eviction at 2,000 entries) records tool invocations with group, status, duration, and cache hit flag. An aggregation API provides period-based summaries (by tool group, by status, average duration, cache hit rate) exposed as an MCP tool (`get_server_metrics`), enabling AI agents to self-diagnose performance issues.

Section 4: Multi-Backend Orchestration Architecture

4.1 Backend Integration Patterns

The gateway integrates seven backend systems through four distinct integration patterns:

Pattern A -- Direct REST Client (Jira): An `httpx`-based asynchronous HTTP client with configurable timeouts, retry logic, and connection pooling. Supports three authentication modes per request: Bearer token (PAT), HTTP Basic (email + API token), or delegated to the enterprise LLM orchestrator. Backend selection is dynamic -- configurable per Jira dashboard instance in DocumentDB -- enabling gradual migration from direct REST to LLM-orchestrated access.

Pattern B -- MCP Sidecar Proxy (Teradata): The Teradata analytical data warehouse is accessed through an open-source MCP server (`teradata-mcp-server`) running as an in-pod sidecar. The gateway performs the full MCP JSON-RPC handshake (initialize, notifications/initialized, tools/call) over HTTP, managing session state with automatic re-handshake on session expiry. Read-only access is enforced at four independent layers:

1. Database account permissions (no write grants)
2. Sidecar profile configuration (`readonly`)
3. Gateway-level SQL validation guardrail (blocks DDL/DML keywords)
4. Row cap enforcement in the gateway client

The sidecar is supervised by the container entrypoint with an exponential backoff restart loop (2s initial, doubling, capped at 60s), ensuring recovery from transient failures without crash-looping the pod.

Pattern C -- Native Driver (Neo4j): The Neo4j knowledge graph is accessed via the official `neo4j` Python driver over the Bolt protocol, with multi-instance support (production and experimentation instances). Read-only access is enforced through regex-based Cypher query validation and automatic `LIMIT` clause injection. Schema introspection results are cached with a configurable TTL.

Pattern D -- Reference Data Service Client (Business Rule Engine): The Business Rule Engine is accessed through a custom REST client that communicates with a Reference Data Management (RDM) API. The client handles the domain-specific workflow: access pre-checks, document retrieval, picklist reference data loading, draft

validation with cross-referencing against cached picklist data, document creation/update, status transitions, and inter-system attachment transfer.

4.2 Enterprise LLM Orchestrator Integration

A notable architectural feature is the gateway's ability to route tool invocations through an enterprise LLM orchestrator (analogous to LiteLLM, Portkey, or PRISMA AIRS). When configured, Jira tool calls are translated from the gateway's internal tool schema to the orchestrator's JSON-RPC interface via a tool name mapping layer. This enables:

- **Centralized AI governance:** The LLM orchestrator applies its own rate limiting, cost tracking, and prompt safety checks
- **Model-agnostic routing:** The orchestrator can select the optimal LLM backend for each request
- **Compliance integration:** Enterprise-wide AI usage metrics are captured in a single pane of glass

The integration supports both synchronous and asynchronous invocation, SSE response parsing for streaming responses, and automatic environment detection from the Jira instance URL.

4.3 Response Caching Architecture

Three cache layers reduce backend load and improve response latency:

Layer 1 -- Redis Distributed Cache: A namespace-aware Redis cache (ElastiCache in production) provides cross-pod response caching. Each tool group has an independent namespace with configurable TTL. The cache only stores successful responses (no error caching). On write operations, targeted invalidation removes affected cache entries (e.g., `cache_invalidate_issue("PROJ-123")` removes all cached responses referencing that issue). Cross-pod cache coordination uses Redis "stamps" -- shared timestamp values that signal cache invalidation events to all pods.

Layer 2 -- Disk-Backed Reference Data Cache: Reference data (picklists, validation rules) is cached to disk using Python's `pickle` serialization, surviving pod restarts. A scheduled daily refresh (06:00 Central time) ensures data freshness, with Redis-based cluster-wide invalidation stamps for on-demand refresh.

Layer 3 -- In-Memory Registry Cache: Tool metadata, consumer profiles, and RBAC roles are cached in thread-safe in-memory structures with TTL-based expiration. When DocumentDB is unavailable, the gateway serves stale cached data rather than failing.

4.4 Decorator Composition and Request Flow

Every tool function follows a consistent decorator composition order, from outermost to innermost:

```
@audit_decorator    -- Captures the full request lifecycle (including blocks)
  @cached           -- Serves cached responses (cache hits bypass rate limiter)
    @rate_limited   -- Enforces two-layer rate limiting
      @guardrail    -- Runs 7 security checks
        tool_logic  -- Actual backend invocation
```

This ordering ensures: - Audit captures everything -- including rate-limited and guardrail-blocked calls - Cache hits do not count against rate limits - Rate limiting is evaluated before expensive guardrail checks - Guardrails are the final gate before backend invocation

4.5 Client Integration: IDE Auth Proxy

For Kiro IDE integration, a zero-dependency Node.js stdio-to-HTTPS proxy bridges the gap between the IDE's MCP stdio transport and the gateway's HTTPS endpoint. The proxy:

1. Manages JWT token lifecycle (reads from credential file, validates expiry with 5-minute buffer)
2. Opens the user's browser to the Kiro Access portal when authentication is required
3. Watches the credential file for updates using `fs.watch` with polling fallback
4. Forwards MCP JSON-RPC messages over HTTPS with authentication headers
5. Automatically re-authenticates on 401/403 responses with a single retry
6. Parses SSE responses from the MCP Streamable HTTP transport

The proxy requires zero external dependencies (no npm install), making it distributable as a single file.

4.6 Stateless HTTP Transport

The gateway configures `stateless_http=True` on the MCP server, removing the protocol's session handshake requirement. This enables plain HTTP consumers (Java Spring `RestTemplate`, Python `httpx`, `curl`) to invoke tools via `POST /mcp` with a JSON-RPC body without performing the MCP `initialize` round-trip. This is critical for

enterprise adoption where existing service consumers cannot adopt the full MCP SDK.

4.7 Feature Flag Architecture

Six CI/CD-driven feature flags enable incremental rollout and safe rollback:

Flag	Default	Controls
JWT_AUTH_ENABLED	false	JWT bearer token validation for service consumers
KIRO_RBAC_ENABLED	false	Gateway-signed token RBAC for IDE consumers
NEO4J_ENABLED	true	Neo4j knowledge graph tool availability
TERADATA_MCP_ENABLED	true	Teradata sidecar and tool availability
CACHE_ENABLED	true	Redis response cache
USER_WRITE_AUTH_ENABLED	true	AD group-based write authorization

Flags are set as CI/CD variables in GitLab, injected into Kubernetes deployment manifests, and read by the gateway at startup. Disabling a flag gracefully degrades the corresponding feature without affecting other functionality.

Section 5: Academic and Industry Mapping

This section explicitly connects the Enterprise MCP Gateway's architectural patterns to graduate-level learning outcomes in natural language processing (NLP), deep learning systems, and distributed AI architectures, supporting Credit for Prior Learning (CPL) portfolio review.

5.1 Natural Language Processing (NLP)

Learning Outcome: Understanding of tool-augmented language models and retrieval-augmented generation (RAG) patterns.

The MCP protocol is the standardized interface through which LLMs access external tools -- the "function calling" capability that transforms a language model from a text generator into an agent capable of taking actions. The Enterprise MCP Gateway sits at the critical juncture between the LLM's intent (expressed as a `tools/call` JSON-RPC message) and the real-world system that fulfills it.

Specific NLP-relevant patterns implemented:

- **JQL Input Validation (Guardrail 4):** The gateway validates Jira Query Language input from AI agents, detecting injection patterns that an LLM might generate when prompted adversarially. This demonstrates applied understanding of the risks of LLM-generated structured queries -- a topic at the intersection of NLP safety and prompt injection research.
- **Cohort-Based Intelligent Defaulting:** The Business Rule Engine's field suggestion system (`bre_get_suggested_default`) applies statistical pattern matching over historical data to predict likely field values given a partial context. While not a neural approach, this implements the same conditional probability estimation that underpins language model next-token prediction, applied to structured form fields rather than natural language tokens.
- **Tool Name Mapping for LLM Orchestrator:** The gateway translates between its internal tool schema and the enterprise LLM orchestrator's expected format, demonstrating the schema alignment challenges that arise when multiple AI systems with different tool interfaces must interoperate -- a practical instance of the "tool specification" problem in agent architectures.

5.2 Deep Learning Systems

Learning Outcome: Applied understanding of production ML/AI system design, including infrastructure, deployment, monitoring, and governance.

The Enterprise MCP Gateway is infrastructure for deep learning systems -- it is the control plane through which deployed AI models interact with enterprise data. Key deep learning system patterns:

- **Multi-Tenant Model Serving Governance:** The gateway's consumer profiles, rate limiting, and tool authorization are directly analogous to model serving infrastructure (e.g., NVIDIA Triton, TensorFlow Serving) that must manage multiple clients with different SLAs. The two-layer rate limiter (per-consumer + per-backend) mirrors the pattern of per-client quotas backed by global GPU compute limits.
- **Feature Flags for Safe Rollout:** The six CI/CD-driven feature flags implement the same incremental rollout pattern used in ML model deployment (canary releases, shadow mode, gradual traffic shifting). Disabling `KIRO_RBAC_ENABLED` is analogous to rolling back a model version -- the system degrades gracefully to the previous behavior.
- **Observability for AI Systems:** The three audit pipelines provide the

observability infrastructure that responsible AI deployment requires. Every tool invocation by an AI agent is logged with the full context needed to debug model behavior: what tool was called, with what parameters, by which model/agent, what was the outcome, and how long did it take. This is the AI-specific extension of MLOps monitoring.

- **Credential Attribution for AI Actions:** Guardrail 3 (Write Credential Attribution) addresses a unique challenge in AI systems: when an LLM agent performs a mutation on behalf of a user, the mutation must be traceable to the human user, not the AI system's service account. This is the "attribution problem" in autonomous AI systems -- ensuring that AI actions are accountable to the humans who authorized them.

5.3 Distributed AI Architectures

Learning Outcome: Design and implementation of distributed systems that support AI workloads at enterprise scale.

The Enterprise MCP Gateway is a distributed system in its own right, and serves as infrastructure for larger distributed AI architectures:

- **Horizontal Scaling with Shared State Coordination:** The gateway runs as 2-4 replicas behind a Kubernetes Horizontal Pod Autoscaler (HPA), with DocumentDB as the shared configuration store and Redis as the shared cache/coordination layer. The 30-second configuration propagation window and Redis stamp-based cache invalidation implement eventual consistency patterns fundamental to distributed systems.
- **Sidecar Pattern for Heterogeneous Backends:** The Teradata integration uses a sidecar pattern -- a separate process in the same pod running its own MCP server, supervised by the container entrypoint with exponential backoff restart. This is a canonical microservice pattern applied to AI tool integration, demonstrating understanding of container orchestration, process supervision, and graceful degradation.
- **Multi-Protocol Gateway:** The gateway bridges four transport protocols (MCP stdio, MCP Streamable HTTP, REST, Bolt) into a unified interface. This is the "API gateway" pattern applied to AI tool ecosystems, demonstrating understanding of protocol translation, connection pooling, and transport-layer concerns in distributed systems.
- **Dynamic Configuration Propagation:** The DocumentDB-backed configuration

plane with bounded TTL cache implements the "externalized configuration" pattern from distributed systems theory. Configuration changes propagate to all replicas within 30 seconds without service restart -- the same pattern used in service mesh control planes (e.g., Istio, Envoy).

- **Circuit Breaker and Fail-Open/Fail-Closed Decisions:** The gateway makes explicit, documented decisions about failure behavior for each component:
 - Audit logging fails open (tool availability over audit completeness)
 - Write authorization fails closed (security over availability)
 - Consumer registry degrades through four tiers (DocumentDB, Secrets Manager, local file, defaults)
 - Redis cache fails open (cache miss, not system failure)

These decisions demonstrate applied understanding of the CAP theorem tradeoffs and the "bulkhead" pattern from distributed systems resilience engineering.

- **Cross-Pod Cache Coordination:** The Redis stamp mechanism for cluster-wide cache invalidation implements a lightweight form of distributed event notification. When one pod invalidates a cache, it writes a timestamp to a shared Redis key. Other pods compare this timestamp against their local cache age on next access, triggering a refresh if needed. This is a practical implementation of the "invalidation-based coherence" protocol from distributed caching theory.

5.4 Summary of Academic Mapping

Graduate Learning Outcome	Gateway Implementation	Demonstrated Competency
Tool-augmented LLM architectures	39 MCP tools with JSON-RPC invocation from AI agents	Design and deployment of LLM tool-use infrastructure
Prompt injection and input safety	JQL/SQL validation guardrails blocking adversarial patterns	Applied NLP security for LLM-generated queries
ML system observability	3 audit pipelines with caller identity, duration, outcome tracking	Production AI monitoring and debugging infrastructure
Model serving governance	Per-consumer rate limiting, tool authorization, RBAC	Multi-tenant AI system resource management

Distributed system design	Horizontal scaling, shared state via DocumentDB/Redis, eventual consistency	Kubernetes-native distributed AI infrastructure
Microservice patterns	Sidecar proxy, circuit breaker, decorator composition	Enterprise microservice architecture for AI backends
AI accountability	Write credential attribution, audit fingerprinting	Responsible AI deployment with human-traceable actions
Feature engineering	Cohort-based BRE field defaulting with statistical pattern matching	Applied statistical inference for structured data prediction
Protocol design	MCP stdio/HTTP bridge, stateless transport, SSE parsing	Multi-protocol AI system integration
Infrastructure as code	CI/CD pipeline, Helm deployment, feature flags, 4 environments	MLOps and AI infrastructure automation

Section 6: Conclusion

6.1 Technical Contribution

This paper has presented the architecture of a production Enterprise MCP Gateway that addresses the fundamental security, governance, and observability gaps in the Model Context Protocol specification. The gateway transforms an inherently open protocol into an enterprise-grade control plane through:

1. **Three-mode authentication** supporting heterogeneous consumer types without protocol modifications
2. **Active Directory group-based RBAC** that scales to organizational scope without per-user provisioning
3. **Seven composable security guardrails** enforced per request, all runtime-configurable via DocumentDB
4. **AES-256-GCM envelope encryption** with AWS KMS for credential storage
5. **Two-layer rate limiting** protecting both individual consumers and shared backend systems
6. **Three parallel audit pipelines** ensuring complete traceability across all tool groups
7. **Multi-backend orchestration** integrating 7 systems through 4 distinct integration patterns

8. DocumentDB as dynamic configuration plane enabling runtime governance without redeployment

The system has been deployed in production across four environments on Amazon EKS, serving 39 tools to multiple consumer classes with measurable security, performance, and compliance characteristics.

6.2 Enterprise Architecture Alignment

The gateway's architecture aligns with enterprise review board standards in several dimensions:

- **Identity:** JWT-based zero-trust authentication with IdP-agnostic claim extraction
- **Authorization:** RBAC with least-privilege scoping, enforced at middleware and guardrail layers
- **Encryption:** AES-256-GCM at rest (KMS), TLS in transit (forced SSL in production)
- **Audit:** Complete request lifecycle logging with credential fingerprinting
- **Resilience:** Documented fail-open/fail-closed policies per component, graceful degradation through multiple fallback tiers
- **Configurability:** Runtime-tunable governance without code deployment
- **Scalability:** Horizontal pod autoscaling with cross-pod state coordination

6.3 Future Work

Several architectural enhancements are planned for subsequent phases:

- **A2A (Agent-to-Agent) Protocol Support:** Extending the gateway to mediate tool access for autonomous agent-to-agent workflows, where one AI agent invokes tools on behalf of another with delegated credentials.
- **SAST/SBOM Integration:** Integrating static application security testing and software bill of materials scanning into the CI/CD pipeline.
- **Container Hardening:** Running the gateway as a non-root user with a read-only filesystem and minimal base image.
- **Network Policies:** Kubernetes NetworkPolicies restricting pod-to-pod communication to authorized paths.
- **AI Hub and CMDB Registration:** Registering the gateway in the enterprise AI Hub and Configuration Management Database for centralized governance.
- **Token Revocation List:** Implementing an explicit token blacklist for immediate

revocation without waiting for TTL expiry.

6.4 Closing Remarks

The Model Context Protocol represents a significant step toward standardized AI-tool integration. However, the protocol's intentional omission of security and governance creates a gap that every enterprise deployer must bridge independently. This paper demonstrates that bridging this gap requires not a single feature but a comprehensive architectural approach -- one that treats security as a composable pipeline, observability as a first-class concern, and runtime configurability as an operational requirement. The Enterprise MCP Gateway described here provides a replicable template for organizations seeking to deploy AI tool access at enterprise scale with the controls that responsible AI governance demands.

This document was prepared for academic portfolio review and enterprise architecture submission. All proprietary system names have been anonymized. Open-source technologies and industry-standard protocols are referenced by their public names.